코드 설명 및 결과 분석 보고서

휴머노이드 예비단원 이주원

1. 서론

2. 데이터 준비
3. 모델 및 학습 설정
4. 미니배치 학습루프
5. 검증 및 테스트 출력

6. 결과 분석

1. 서론

이 코드는 크게 총 4개의 부분으로 나누어져 있다. 첫번쨰로 데이터를 준비하는 부분이다. 여기서는 필요한 라이브러리를 import하고 학습에 쓰일 데이터를 불러온다. 두번째로 모델 및 학습을 설정한다. 이 부분에서는 모델을 정의하고 학습에 쓰이는 각종 파라미터들을 정의한다. 세번째로 미니배치에 대한 학습 루프를 구현한다. 네번쨰는 검증 및 테스트 결과 출력부분이다. 학습한 모델에 대해 얼마나 정확하고 손실비용을 갖는지를 출력한다.
그리고 코드에 대한 설명이 끝난뒤 결과에 대한 분석 보고서를 작성하였다. 결과는 같이 첨부된 RLDay1log.pdf 파일을 분석한 것이다.

*실행방법은 README파일을 참고

2. 데이터 준비

import torch 를 사용하여 이 과제의 핵심인 pytorch를 불러온다. 또한 여러 라이브러리를 임포트하여 코드의 편의성을 높였다.

32줄 부터 51줄은 MNIST 데이터셋을 읽어서 pytorch가 쓸 수 있는 데이터셋으로 만든다. load_idx_images를 이용해서 이미지를 읽고, load_idx_labels로 라벨을 읽는다.
mnist_train은 학습용 데이터 6만장, mnist_test는 테스트용 데이터 1만장으로 한다.

53~57줄
```
# dataset loader
data_loader = torch.utils.data.DataLoader(...)
```
앞서 만든 mnist_train 6만장을 배치 단위로 꺼내어주는 역할이다.
괄호 안은 주석의 설명대로 dataset부분은 어떤 데이터를 사용할건지 적고 배치 사이즈는 한 번에 몇 개를 들고 올건지, shuffle은 무작위로 불러올건지, drop_last는 예를 들어 100개식 들고 올때 남는게 있으면 드랍(버림)할 건지 정한다.

3. 모델 및 학습 설정

21줄부터 24줄은 모델에 대한 파라미터 정이다 에포크와 배치 사이즈, 학습률을 설정할 수 있다. 비교적 앞에 코드를 두어 사용자가 편하게 바꿀 수 있게 했다.

59~64줄:
```
model = nn.Sequential(...)
```
모델을 정의하는 부분이다. 28*28이미지를 받아서 0에서9중 각 숫자의 점수를 내보낸다. model(X)를 하면 Flatten Linear ReLU Linear순으로 지나간다.

66~67줄:
criterion...: 손실함수 점수10개에 대해 softmax를 적용해서 확률로 바꿔주고, 정답 클래스 확률에 -log를 취한다.
optimizer...: 경사하강법 SDG는 가장 기본적인 경사하강법이다.

4. 미니배치 학습루프

실제로 학습이 일어나는 부분이다. 앞에서 만든 데이터 로더, 모델, 손실함수, 옵티마이저를 다 가져와서 반복해서 가중치를 설정한다.

```
for epoch
        for 배치
```
의 이중 반복문을 갖는다. 예시로 배치 사이즈가 100이고 에포크가 15이면 총 15 * 600 = 9000번의 가중치 업데이트가 일어난다.

optimizer.zero_grad()를 이용해서 이전 배치의 기울기를 지운다. 안 지우면 계속 기울기가 누적되기 때문에 꼭 필요한 작업이다.
logits = model(X)는 순전파 점수가 나온다.
loss = criterion(logits, Y)는 배치 100장의 평균 loss를 계산한다.
loss.backward()역전파를 계산한다.
optimizer.step()으로 실제 가중치를 바꾼다.

5. 검증 및 테스트 출력

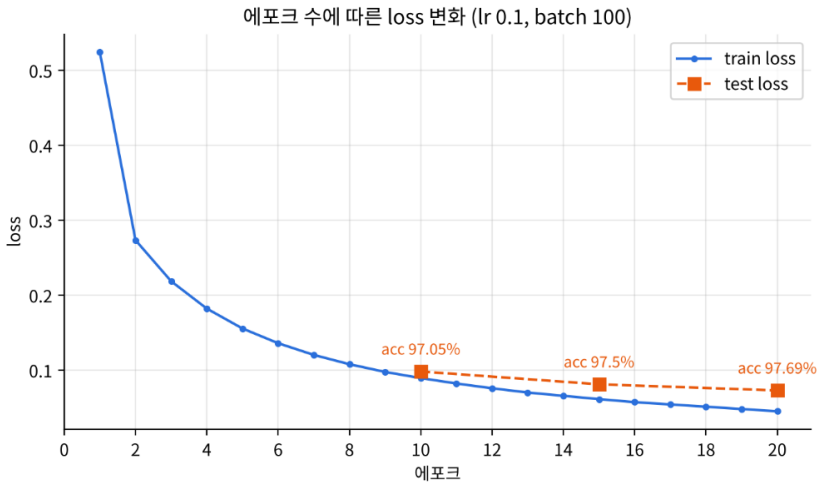
학습이 끝난 모델을 테스트 데이터 1만장으로 검증하는 부분이다.
prediction = model(X_test) 에서 1만장을 예측한다.
test_loss = criterion(prediction, Y_test)테스트셋 평균 loss이다. 마지막 에포크의 train loss와 비교한다.

6. 결과 분석

6-1 실험 조건

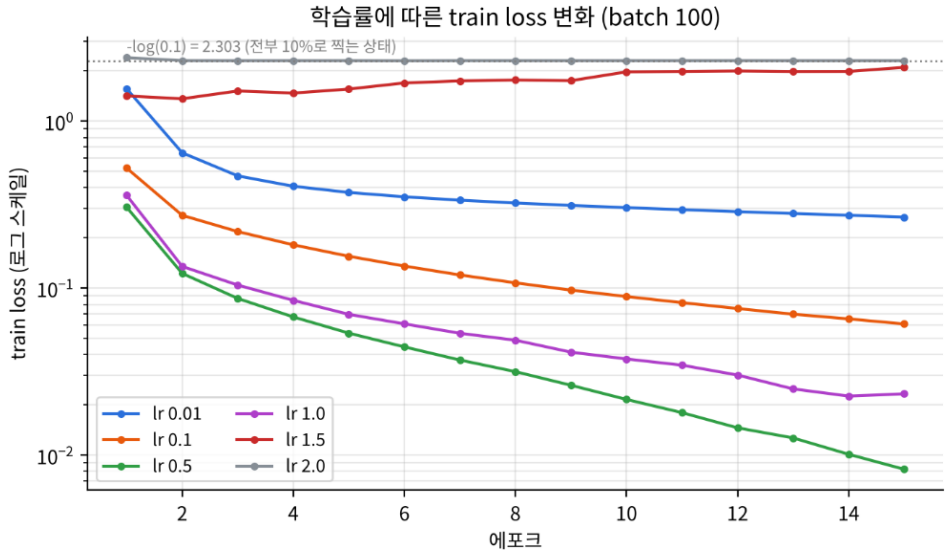
실험을 하기전 변인 통제를 하는것이 가장 중요하다고 생각했다. 따라서 실험의 조건을 다음과 같이 고정하고 에포크, 학습률, 배치를 다르게 해서 실험을 진행했다.
먼저 시드를 777로 고정하고 모델의 구조를 동일하게 유지 하였으며 SDG만을 사용하였다. 실험에 대한 예시를 들어보면 에포크를 변화하고 싶으면 학습률과 배치를 고정하고, 또 학습률을 변하게 하고 싶으면 에포크와 배치를 고정하는식으로 진행하였다.

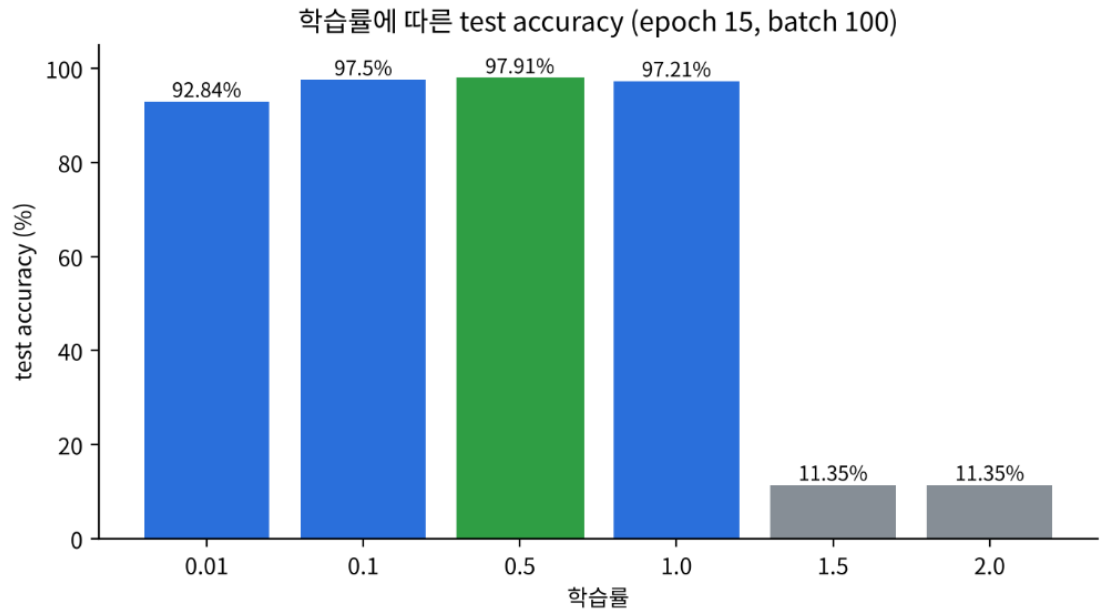
6-2 에포크 변화에 따른 결과 차이



처음 1~2에포크에서는 loss가 눈에 띄게 변화하는 모습이 관찰된다. 하지만 그 뒤로느 완만하게 줄어드는데, 이유를 생각해본다면 처음에는 가중치가 랜덤이라 경사가 크고 수정사항이 많아서 크게 움직이는데, 바닥 근처로 갈수록 경사가 작아져 적게 움직이는 것이다. 그래프를 보면 10이상의 구간에서 train loss의 변화보다 test loss가 더 적게 변화하는 것을 관찰할 수 있는데 이는 일반적인 숫자의 모양보다 학습데이터에 좀 더 잘 맞춰지고 있는것으로 보인다. 에포크가 20까지 테스트 해봤을 때 아직 까지는 과적합의 경향은 보이고 있지 않지만 에포크를 더 늘린다면 충분히 과적합의 경향을 보일 것으로 예측된다.

6-3 학습률 변화에 따른 결과 분석





여기서는 각각의 학습률을 따져가며 분석한다.

0.01: 변화를 너무 적게 주어 15 에포크 동인 경사의 밑바닥 까지 충분히 못 간 것으로 보인다. 아직까지 곡선이 내려가는 중이라 에포크를 좀 더 늘리면 좋아지ㅣㄹ 것으로 보인다.

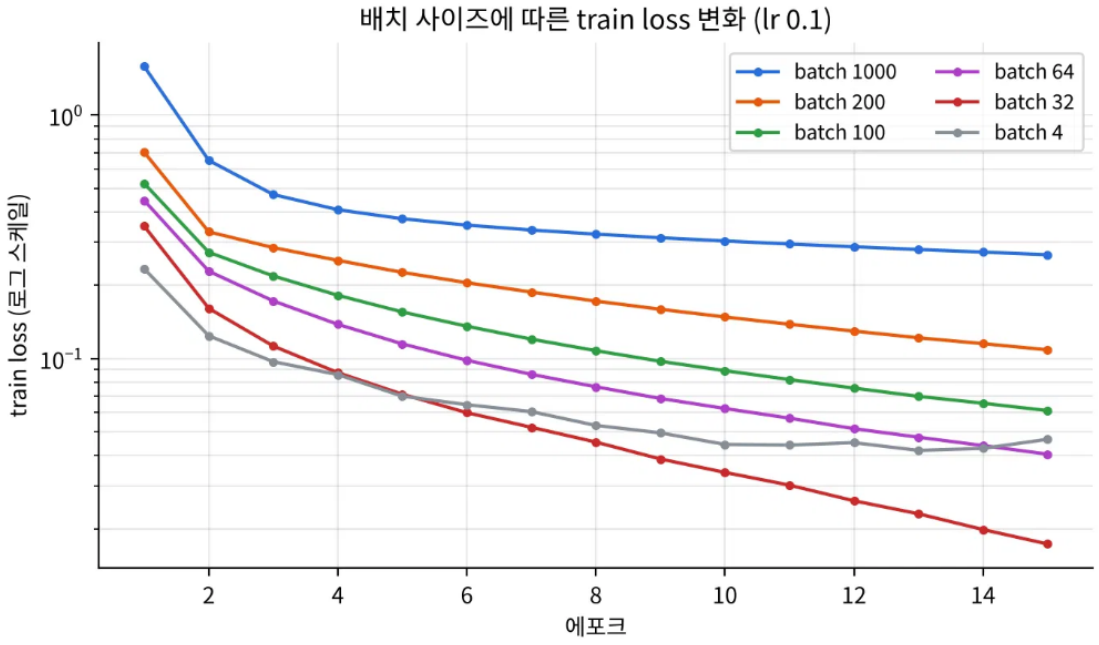
0.1 -> 0.5: 변화가 커지면서 같은 에포크 안에서 더 깊이 내려갔다. 같은 실험 조건에서 학습률 0.5가 train loss 0.008, 정확도 97.91 퍼센트로 가장 적합한 학습률을 기록했다.

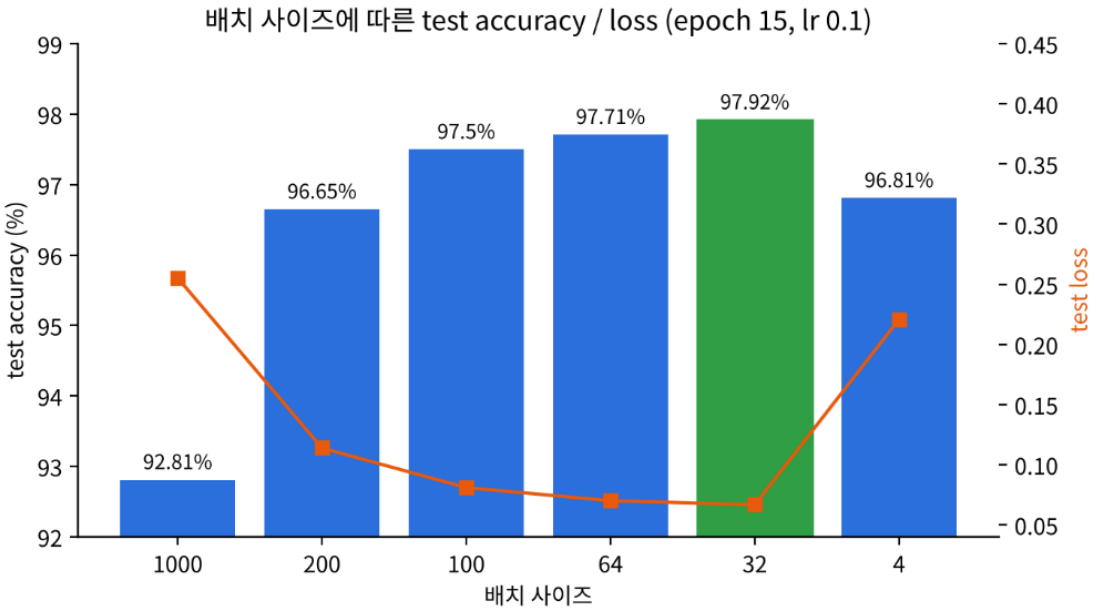
1.0: 처음에는 빨리 하강하지만 14->15에포크에서 loss가 다시 살짝 올라간다. 변화 폭이 커서 골짜기를 왔다갔다 하는 것처럼 보인다. train loss 는 낮지만 test loss는 0.124로 0.5보다 커서 오히려 불안정하게 학습된것 같다.

1.5: 첫 에포크 부터 loss가 1.4로 높고 갈수록 증가하는 경향을 보인다. 에포크가 증가할 때마다 최소점을 넘어버려 점점더 멀어지는 것으로 분석할 수 있을거 같다.

2.0: 첫 에포크 부터 이미 도리킬 수 없이 망가져서 loss가 2.3부근에 고정되어 있는 것을 볼 수 있다. 너무 큰 업데이트 때문에 은닉층 ReLU 뉴런들이 전부 음수 영역으로 밀려나서 출력이 0이 되고, 기울기도 0이라 다시 학습이 안되는 상태이다. 출력이 사실상 상수가 되어서 모든 이미지를 같은 숫자로 예측하여 정확도가 숫자 1의 비율인 약 10%가 나온것이다.

6-4 배치 크기 변화에 따른 결과 분석





1000: 에포크당 60번 밖에 업데이트를 안 하였다. 이는 기울기는 안정적으로 나타나지만 움직이는 횟수가 적어 덜 학습됐다고 볼 수 있다.

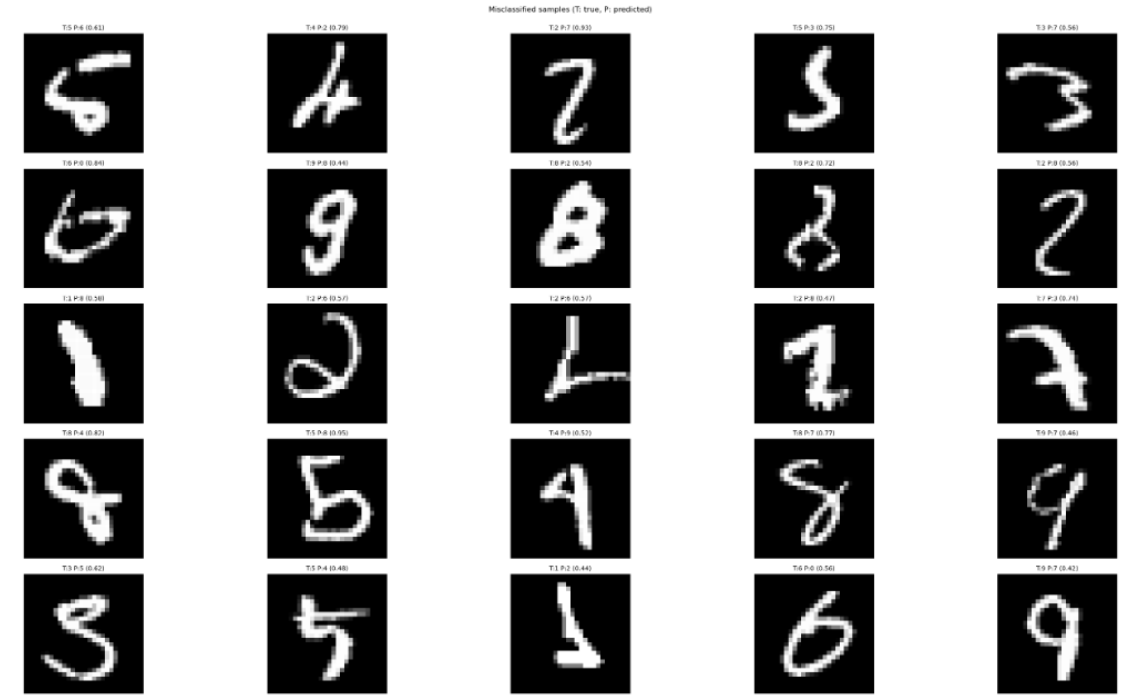
200 -> 32: 배치가 작아질 수록 업데이트 횟수가 늘어서 같은 에포크 안에 더 많은 학습이 이루어진다. 이 때문에 정확도는 오르는 추세를 보인다.

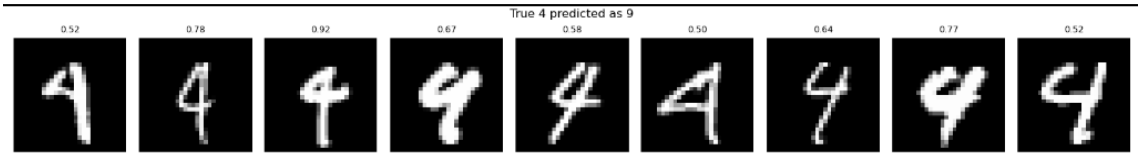
4: 가중치를 수정하는 횟수는 실험 중에서 가장 많지만 전체 데이터의 방향을 거의 대표하지 못해서 노이즈가 심하다. 그래서 곡선이 10 에포크 이후로 평평해지다가 오르락 내리락 하고, test loss도 0.22로 커지는 이유중 하나 이다.

6-5 전체적인 분석

학습률과 배치 사이즈 모두 극단적인 값들로 설정하면 결과가 불안정해지는 패턴이 나온다. 두 값은 서로 얽혀 있어서 두 개의 값을 적당한 중간값에서 조절하는 것이 가장 성능이 이상적으로 나온다고 본다.

6-6 오분류 검증에 대한 분석





오분류 사례를 나타낸것이다. 유형들을 분석해보자면 4를 9로 판단하는 대부분은 닫힌 고리의 유무로 숫자를 구분하는것 같은데 위 사례의 대부분에서 바뀐 그림이라 그런것 같다. 또한 획이 너무 두꺼워 픽셀의 분포가 다른 숫자와 비슷해지는 것 또한 이유로 들 수 있을것 같다. 2를 7로 보는 경우나, 5를 3으로 보는 경우에서 보이듯이 가로획이나 받침 하나의 차이로 다른 숫자와의 특징이 겹쳐서 발생한 유형으로 보인다. 그리고 사람도 햇갈리는 경우도 있을수 있다. 라벨과 모양자체가 애매한 경우 판단하기 애매하다. 원인이 불명확한 경우도 있을수 있다. 이는 MLP의 구조적 한계로 보인다.